COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/ Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	August 4, 2026	**DATE SUBMITTED:**	August 11, 2026
NAME:	Carl Andrei D. Torres	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 **Hands-On Activity 5.1 Queues**

1. PROCEDURE OUTPUT

ILO A

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> r);

int main(){

    std::queue<char> q;

    q.push('c');
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('1');
    q.push('0');

    //Display
    display(q);

    //empty()
    std::cout<<"Is the stack Empty? "<<q.empty()<<std::endl;
    //size()
    std::cout<<"The size of the queue is: " << q.size() <<std::endl;

    //front
    std::cout<<"The front of the queue is: " << q.front()<<std::endl;

    //back
    std::cout<<"The back of the queue is: " <<q.back()<<std::endl;

    //pop
    q.pop();

    //display
    display(q);

    //push another element
    q.push('C');
    q.push('A');
    q.push('R');
    q.push('L');

    //display
    display(q);
}
```

```
46
47  void display(std::queue<char> r){
48      //Create a copy of the queue
49      std::queue<char> c = r;
50
51      while(!c.empty()){
52          std::cout<<" "<<c.front();
53          c.pop();
54      }
55      std::cout<<"\n";
56  }
```

MAIN DRIVER CODE

```
 c p e 0 1 0
Is the stack Empty? 0
The size of the queue is: 6
The front of the queue is: c
The back of the queue is: 0
 p e 0 1 0
 p e 0 1 0 C A R L
```

OUTPUT

Code Analysis	In this code, it is a new type of implementation in queue which the “queue” library was used to utilize the q.push, q.pop, q.empty, q.size, q.front, and q.back. Also, I’ve created a custom function which the Display Function which we create a copy of the queue first by std::queue<char> c = r to easily pop the front then display the new front again and again by while loop.

ILO B 1

```cpp
#include <iostream>
#include "queuell.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    std::cout<<"---ENQUEUE---\n";
    enqueue(&front, &back, 'C');
    std::cout<< front -> data << " " << back -> data << " " << std::endl;
    enqueue(&front, &back, 'A');
    std::cout<< front -> data << " " << back -> data << " " << std::endl;
    enqueue(&front, &back, 'T');
    std::cout<< front -> data << " " << back -> data << " " << std::endl;
    std::cout<<"---DEQUEUE---\n";
    dequeue(&front, &back);
    std::cout<< front -> data << " " << back -> data << " " << std::endl;
    std::cout<<"---DISPLAY LIST---\n";
    display(&front, &back);
    std::cout<<"---DISPLAY FRONT---\n";
    std::cout<< "The Front of the Queue is: " << FrontQ(front) << "\n";


    return 0;

}
```

Main Driver Code

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

template <typename T>
class qNode{
    public:
    T data;
    qNode *next;
};

//creating a new node
template <typename T>
qNode<T>* new_node(T newData){
     qNode<T>* newNode = new qNode <T>;
     newNode -> data = newData;
     newNode -> next = nullptr;
     return newNode;
}

template<typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData){
    //Create a new node
    qNode<T>* newNode = new_node(newData);

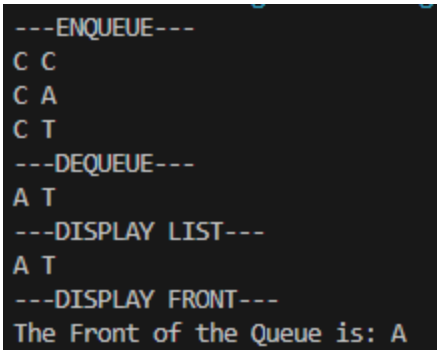
    //Inserting to an empty queue
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
    (*frontPtr) = newNode;
    (*backPtr) = newNode;
    }

    //Inserting an item into a non empty queue
    //Point the backPtr next to the newNode
    (*backPtr) -> next = newNode;
    (*backPtr) = newNode;
}

template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
     //create a temporary node to store the node to be deleted
     qNode<T>* deleteNode = nullptr;
     deleteNode = (*frontPtr);
     //check if the queue is only one node
     if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
```

```
48          }
49          //deleting of the node
50          (*frontPtr) = deleteNode -> next;
51          deleteNode -> next = nullptr;
52          delete deleteNode;
53      }
54
55      //display all the elements in the list
56      template <typename T>
57      void display(qNode<T>** frontPtr, qNode<T>** backPtr){
58          qNode<T>* temp = (*frontPtr);
59
60          if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
61              std::cout<<"The Queue is Empty!\n";
62              return;
63          }
64
65          do {
66              std::cout<< temp -> data << " ";
67              temp = temp->next;
68          }while(temp!= nullptr);
69          std::cout << "\n";
70      }
71
72      //return the front variable
73      template <typename T>
74      T FrontQ(qNode<T>* frontPtr){
75          if  (frontPtr == nullptr) {
76              throw std::runtime_error("The Queue is Empty");
77          }
78          return frontPtr->data;
79      }
80
81
82
83      #endif
```

Header File



OUTPUT

Code Analysis	In the Linked List implementation of the queue, it is the same like it has main driver and header file but the implementation of the operations is different like in the display the front which you need to throw the data and check if the frontPtr is empty. If the frontPtr is not empty, then it will display the front variable through the main code.

ILO B 2

queueArr > C queueArr.h > ~Queue_Arr()

```h
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>

template <typename T>
class Queue_Arr {
private:
    T* queue_Arr; // Pointer to dynamically allocated array for queue elements
    int front;    // Index of the front element in the queue
    int back;     // Index of the back element in the queue
    int capacity; // Maximum number of elements the queue array can hold
    int size;     // Current number of elements in the queue

public:
    // Prototypes:
    Queue_Arr(size_t cap);
    bool Empty() const;
    bool Full() const;

    size_t Size() const;
    void Clear();

    T Front() const;
    T Back() const;

    void Enqueue(T newData);
    void Dequeue();
    void Display() const;

    // Rule of Three
    ~Queue_Arr();
    Queue_Arr(const Queue_Arr& other);
    Queue_Arr& operator=(const Queue_Arr& other);
};
```

```cpp
// --- Method Definitions ---

//Constructor
template <typename T>
Queue_Arr<T>::Queue_Arr(size_t cap) { // Initializes the queue with a specified capacity
    capacity = static_cast<int>(cap); // Allocates memory for the queue array based on the specified capacity
    queue_Arr = new T[capacity]; // Dynamically allocates an array of type T with the specified capacity
    front = 0;
    back = -1;
    size = 0;
    std::cout << "Constructor was called" << std::endl;
}

// Size Method
template <typename T>
size_t Queue_Arr<T>::Size() const {
    return static_cast<size_t>(size); // Returns the current number of elements in the queue
}

// Empty Method
template <typename T>
bool Queue_Arr<T>::Empty() const {
    return size == 0;
}

// Full Method
template <typename T>
bool Queue_Arr<T>::Full() const {
    return size == capacity;
}

// Clear Method
template <typename T>
void Queue_Arr<T>::Clear() {
    front = 0;
    back = -1;
    size = 0;
}
```

```cpp

// Front Accessor
template <typename T>
T Queue_Arr<T>::Front() const {
    if (Empty()) {
        std::cout << "Queue is empty\n";
        return T{};
    }
    return queue_Arr[front];
}

// Back Accessor
template <typename T>
T Queue_Arr<T>::Back() const {
    if (Empty()) {
        std::cout << "Queue is empty\n";
        return T{};
    }
    return queue_Arr[back];
}

// Enqueue Method
template <typename T>
void Queue_Arr<T>::Enqueue(T newData) {
    if (Full()) {
        std::cout << "Queue is full\n";
        return;
    }
    back = (back + 1) % capacity;
    queue_Arr[back] = newData;
    size++;
}

// Dequeue Method
template <typename T>
void Queue_Arr<T>::Dequeue() {
    if (Empty()) {
        std::cout << "Queue is empty\n";
        return;
```

```
    }
    front = (front + 1) % capacity;
    size--;
}

// Display Method
template <typename T>
void Queue_Arr<T>::Display() const {
    if (Empty()) {
        std::cout << "Queue is empty\n";
        return;
    }
    for (int i = 0; i < size; i++) {
        std::cout << queue_Arr[(front + i) % capacity] << " ";
    }
    std::cout << std::endl;
}

// Copy Constructor
template <typename T>
Queue_Arr<T>::Queue_Arr(const Queue_Arr& other) {
    capacity = other.capacity;
    size = other.size;
    front = 0; // Initializes front index to 0 for the new queue
    back = (size == 0) ? -1 : size - 1; // Initializes back index based on size
    queue_Arr = new T[capacity];

    // Copies elements from the other queue to this queue, maintaining the correct order
    for (int i = 0; i < size; i++) {
        queue_Arr[i] = other.queue_Arr[(other.front + i) % other.capacity];
    }
    std::cout << "Copy Constructor was called" << std::endl;
}

// Copy Assignment Operator
template <typename T>
Queue_Arr<T>& Queue_Arr<T>::operator=(const Queue_Arr& other) {
```

```
Queue_Arr<T>& Queue_Arr<T>::operator=(const Queue_Arr& other) {
    if (this == &other) {
        return *this;
    }
    delete[] queue_Arr; // Deallocates existing memory to prevent memory leaks

    // Allocates new memory and copies the elements from the other queue
    capacity = other.capacity;
    size = other.size;
    queue_Arr = new T[capacity];

    // Copies elements from the other queue to this queue, maintaining the correct order
    for (int i = 0; i < size; i++) {
        queue_Arr[i] = other.queue_Arr[(other.front + i) % other.capacity];
    }
    front = 0; // Initializes front index to 0 for the new queue
    back = (size == 0) ? -1 : size - 1; // Initializes back index based on size

    std::cout << "Copy Assignment Operator was called" << std::endl;
    return *this; // Returns the current object to allow for chained assignments
}

// Destructor
template <typename T>
Queue_Arr<T>::~Queue_Arr() { // Destructor to clean up dynamically allocated memory
    delete[] queue_Arr; // Deallocates the dynamically allocated memory for the queue array
    std::cout << "Destructor was called" << std::endl;
}

#endif
```

HEADER FILE

```cpp
#include <iostream>
#include "queueArr.h"

int main() {
    std::cout << "=== 1. Creating Queue Instance (Capacity: 5) ===" << std::endl;
    Queue_Arr<std::string> q1(5);

    std::cout << "\n=== 2. Checking Initial State ===" << std::endl;
    std::cout << "Is the queue empty? " << q1.Empty() << std::endl;
    std::cout << "Size of the queue: " << q1.Size() << std::endl;

    std::cout << "\n=== 3. Enqueuing Elements ===" << std::endl;
    q1.Enqueue("C");
    q1.Enqueue("A");
    q1.Enqueue("R");
    q1.Enqueue("L");
    q1.Enqueue("T");

    // Attempting to push beyond capacity
    q1.Enqueue("R");
    std::cout << "Attempted to enqueue beyond capacity." << std::endl;

    std::cout << "\n=== 4. Displaying Queue Content ===" << std::endl;
    std::cout << "Queue elements: " << std::endl;
    std::cout << "Front of the queue: " << q1.Front() << std::endl;
    std::cout << "Back of the queue: " << q1.Back() << std::endl;
    std::cout << "Size of the queue: " << q1.Size() << std::endl;

    std::cout << "\n=== 5. Testing Copy Constructor ===" << std::endl;
    Queue_Arr<std::string> q2(5); // Create a new queue with the same capacity
    std::cout << "The Copy Constructor was called" << std::endl;

    std::cout << "\n=== 6. Testing Copy Assignment Operator ===" << std::endl;
    q2 = q1; // Triggers Copy Assignment Operator
    std::cout << "The Copy Assignment Operator was called" << std::endl;

    std::cout << "\n=== 7. Dequeuing Elements from Original Queue ===" << std::endl;
    q1.Dequeue();
    q1.Dequeue();
    std::cout << "Queue elements after 2 dequeues: " << std::endl;
    std::cout << "The front element is: " << q1.Front() << std::endl;
    std::cout << "The back element is: " << q1.Back() << std::endl;
```

```
43
44      std::cout << "\n=== 8. Circular Enqueue Behavior ===" << std::endl;
45      q1.Enqueue("T");
46      q1.Enqueue("RRS");
47      std::cout << "Queue elements after wrap-around enqueue: " << std::endl;
48      std::cout << "The front element is: " << q1.Front() << std::endl;
49      std::cout << "The back element is: " << q1.Back() << std::endl;
50
51      std::cout << "\n=== 9. Testing Clear Method ===" << std::endl;
52      std::cout << "Clearing the queue..." << std::endl;
53      q1.Clear();
54      std::cout << "Is the queue empty after clear? " << q1.Empty() << std::endl;
55      std::cout << "Size of the queue after clear: " << q1.Size() << std::endl;
56
57      std::cout << "\n=== 10. Exiting Scope (Destructors Auto-Called) ===" << std::endl;
58      return 0;
59  }
```

MAIN DRIVER FILE

```
=== 1. Creating Queue Instance (Capacity: 5) ===
Constructor was called

=== 2. Checking Initial State ===
Is the queue empty? 1
Size of the queue: 0

=== 3. Enqueuing Elements ===
Queue is full
Attempted to enqueue beyond capacity.

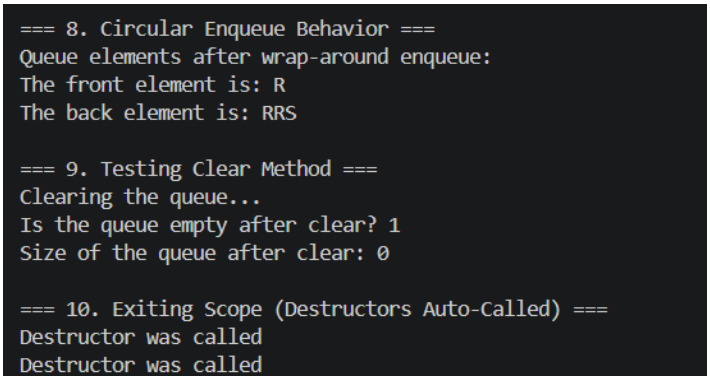
=== 4. Displaying Queue Content ===
Queue elements:
Front of the queue: C
Back of the queue: T
Size of the queue: 5

=== 5. Testing Copy Constructor ===
Constructor was called
The Copy Constructor was called

=== 6. Testing Copy Assignment Operator ===
Copy Assignment Operator was called
The Copy Assignment Operator was called

=== 7. Dequeuing Elements from Original Queue ===
Queue elements after 2 dequeues:
The front element is: R
The back element is: T

=== 8. Circular Enqueue Behavior ===
Queue elements after wrap-around enqueue:
The front element is: R
The back element is: RRS
```



OUTPUT

CODE and OUTPUT ANALYSIS	In this Code, **Queues in Array differ from Linked List due to the presence of the Circular Modulo and the q.size is static as per the characteristics of Arrays**. The **Queue header file comprises eleven functions for initializing, destructing, enqueuing, dequeuing, displaying, clearing, and getting the size and front and back elements of the queue**. **The Queue(int cap) constructor initializes the array and sets the capacity, front = 0, back = -1, and size = 0. The ~Queue() destructor deallocates the array to avoid memory leaks**. The **enqueue(T newData) function adds a new element to the queue by incrementing the back pointer to (back + 1)%capacity and incrementing the size if there is space**. The **dequeue() function deletes the front element by incrementing the front pointer to (front + 1)%capacity and decrementing the size**. The **clear() function sets the front and back pointers to 0 and -1, respectively, and the size to 0 to clear the queue**. The **isEmpty() function checks if size equals 0 and prints the result**. The **getSize() function returns the number of elements in the queue**. The **getFront() function prints the front element of the queue**, and the **getBack() function prints the back element of the queue**. The **display() function prints the elements of the queue in order using a loop that starts from (front + i)%capacity, where i ranges from i = 0 to size – 1**. The **assignment operator copies the elements of the other queue to the current queue by first checking if the current queue is not self-assigned, deleting the array, allocating a new array, and copying the elements one by one from the other queue's array to the current queue's array at positions 0 to size – 1 and returns the current queue**. In the **main driver file and output, the program runs in a sequential manner and comprises nine sections**. **In section 1, the Queue q1(5) creates an instance of the queue with a capacity of 5 and prints the message "Constructor was called". In section 2, the calls to q1.isEmpty() and q1.getSize() check if the queue is empty and its size, which prints the message "Queue is empty" and "Size of the queue: 0". In section 3, the enqueue operation that adds six strings ("C", "A", "R", "L", "T", "R") to the queue checks the overflow safety and prints the message "Queue is full". In**

	section 4, the calls to q1.display(), q1.getFront(), q1.getBack(), and q1.getSize() print "Queue elements: C A R L T", "Front of the queue: C", "Back of the queue: T", and "Size of the queue: 5". **In section 5, the declaration of another queue, Queue q2(5), creates another instance of the queue with a capacity of 5, which prints the message "Constructor was called" again, and the assignment q2 = q1 copies the contents of q1 to q2, which prints the message "The Copy Assignment Operator was called" and "C A R L T"**. **In section 6, the two calls to q1.dequeue() delete the first two elements of the queue ("C" and "A"), which makes it possible to print "Queue elements after 2 dequeues: R L T" and "Front of the queue: R"**. **In section 7, the enqueue operations for elements "T" and "RRS" demonstrate the wrap around of the array and print the message "Queue elements after wrap-around enqueue: R L T T RRS"**. **In section 8, the call to q1.clear() clears the queue and prints the message "Queue is empty" and "Size of the queue: 0"**. Finally, **in section 9, the end of the main function causes q1 and q2 to go out of scope and destroy the objects, thus calling the destructor and printing the message "Destructor was called"**. In this way, all the operations are performed sequentially and produce the expected and organized results.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#ifndef CLASSJOB_H
#define CLASSJOB_H

#include <iostream>
#include <string>

template <typename T>
class Job {
private:
    int id;
    std::string userName;
    int numPages;

public:
    Job();
    Job(int jobId, std::string user, int pages);

    int getId() const;
    std::string getUserName() const;
    int getNumPages() const;

    void print() const;
};

// Default Constructor
template <typename T>
Job<T>::Job() : id(-1), userName(""), numPages(0) {}

// Parameterized Constructor
// Auto Incremented ID, name of the user request to print the job, and the count of pages in the document
template <typename T>
Job<T>::Job(int jobId, std::string user, int pages)
    : id(jobId), userName(user), numPages(pages) {}

// Member Functions

//Retrieves the job's unique identifier.
//Integer represents the Job ID
template <typename T>
int Job<T>::getId() const {
    return id;
}
```

```cpp

//Retrieves the username associated with the job
// std::string containing the username
template <typename T>
std::string Job<T>::getUserName() const {
    return userName;
}

//Retrieves the total number of pages in the job
//Integer for page count
template <typename T>
int Job<T>::getNumPages() const {
    return numPages;
}

// Outputs the Printed Prompt of the details of the job to standard output .
// Format: " [Job #<id>] User: <userName> | Pages: <numPages>"
template <typename T>
void Job<T>::print() const {
    std::cout << " [Job #" << id << "] User: " << userName << " | Pages: " << numPages << std::endl;
}

#endif
```

classJob HEADER FILE

```cpp
#ifndef PRINTER_H
#define PRINTER_H

#include <iostream>
#include <string>
#include "classJob.h"

//Circular Array Queue
class Printer {
private:
    static const int CAPACITY = 100;  // max pending jobs the Printer can hold
    Job<int> jobs[CAPACITY];
    int frontIdx;   // index of the job that will be served next
    int rearIdx;    // index where the next job will be inserted
    int count;      // number of jobs currently queued
    int nextJobId;  // auto-incrementing job id generator

public:
    // Prototypes
    // Constructor specifying max capacity of spooler queue
    Printer(size_t capacity = 100);

    // Queue status checks
    bool isEmpty() const;
    bool isFull() const;
    int pendingCount() const;

    // Queue Operations
    void addJob(const std::string& userName, int numPages);
    void processNextJob();
    void processAllJobs();

    // Utility
    void showQueue() const;
};

// Method Definitions *Inline Operations for the compiler to copy-paste the function's class body directly into the place where it was called*

inline Printer::Printer(size_t capacity) : frontIdx(0), rearIdx(0), count(0), nextJobId(1) {}

//Checks if the Queue is empty
inline bool Printer::isEmpty() const {
```

```cpp
    return count == 0;
}

//Checks if the Queue is full
inline bool Printer::isFull() const {
    return count == CAPACITY;
}

//Checks the current pending job count
inline int Printer::pendingCount() const {
    return count;
}

//Enqueue a Job
inline void Printer::addJob(const std::string& userName, int numPages) {
    //Check if the Queue is Full
    if (isFull()) {
        std::cout << "  [Printer] Queue is full! Cannot accept job from "
                  << userName << "." << std::endl;
        return;
    }

    // Instantiation of the job with a unique ID and inserts  it at the rear position
    Job<int> newJob(nextJobId++, userName, numPages);
    jobs[rearIdx] = newJob;
    // Advance rear index using Circular Modulo Operation
    rearIdx = (rearIdx + 1) % CAPACITY;
    count++;

    //Printer Prompt
    std::cout << "  [Printer] Job #" << newJob.getId() << " from "
              << userName << " (" << numPages << " pages) added to queue. "
              << "Pending jobs: " << count << std::endl;
}

//Dequeue a Job
inline void Printer::processNextJob() {
    // Checks if the queue is empty
    if (isEmpty()) {
```

```
81      if (isEmpty()) {
82          std::cout << "  [Printer] No jobs to process. Printer is idle." << std::endl;
83          return;
84      }
85
86      //Retrieves the job located at the front of the queue
87      Job<int> current = jobs[frontIdx];
88      //Advance front index using the circular modulo and update the job count
89      frontIdx = (frontIdx + 1) % CAPACITY;
90      count--; //decrement the count
91
92      //Printer Prompt
93      std::cout << "  [Printer] Now printing Job #" << current.getId()
94                << " for " << current.getUserName() << " ("
95                << current.getNumPages() << " pages)... Done." << std::endl;
96  }
97
98  // Processes every pending jobs in the queue sequentially until the queue is empty
99  inline void Printer::processAllJobs() {
100     std::cout << "\n-- Printer processing all pending jobs (FIFO) --" << std::endl;
101     if (isEmpty()) {
102         std::cout << "  [Printer] Nothing to print." << std::endl;
103         return;
104     }
105
106     //Continuously dequeue jobs until count reaches to zero
107     while (!isEmpty()) {
108         processNextJob();
109     }
110     std::cout << "-- All jobs processed. Printer is idle. --\n" << std::endl;
111 }
112
113 //Display all queued jobs from front to back queue format
114 inline void Printer::showQueue() const {
115     std::cout << "  Current queue for the job (" << count << " job(s)), front -> back:" << std::endl;
116     if (isEmpty()) {
117         std::cout << "    (empty)" << std::endl;
118         return;
119     }
120
121     // Traverse starting from the front index for 'count' iterations using the modulo operator
122     int idx = frontIdx;
123     for (int i = 0; i < count; i++) {
124         jobs[idx].print();
125         idx = (idx + 1) % CAPACITY;
126     }
127 }
128
129 #endif
```

classPrinter HEADER FILE

```cpp
#include <iostream>
#include "classJob.h"
#include "classPrinter.h"

int main() {
    Printer office_Printer;

    std::cout << "=== Printer Simulation ===\n" << std::endl;

    std::cout << "Morning: several employees send jobs to the shared printer.\n";
    office_Printer.addJob("Carl", 7);
    office_Printer.addJob("Jerry", 12);
    office_Printer.addJob("Roy", 5);

    std::cout << std::endl;
    office_Printer.showQueue();

    std::cout << "\nThe printer starts working through the queue one job at a time.\n";
    office_Printer.processNextJob();   // Alice's job finishes first (FIFO)

    std::cout << "\nWhile the printer was busy, there are more jobs arriving:\n";
    office_Printer.addJob("Ryo", 3);
    office_Printer.addJob("Joaquin", 8);

    std::cout << std::endl;
    office_Printer.showQueue();

    std::cout << "\nThe printer keeps going until the queue is empty:\n";
    office_Printer.processAllJobs();

    std::cout << "One more job arrives after everything cleared out:\n";
    office_Printer.addJob("Kylian", 2);
    office_Printer.processAllJobs();

    return 0;
}
```

MAIN CODE

```
=== Printer Simulation ===

Morning: several employees send jobs to the shared printer.
  [Printer] Job #1 from Carl (7 pages) added to queue. Pending jobs: 1
  [Printer] Job #2 from Jerry (12 pages) added to queue. Pending jobs: 2
  [Printer] Job #3 from Roy (5 pages) added to queue. Pending jobs: 3

  Current queue for the job (3 job(s)), front -> back:
 [Job #1] User: Carl | Pages: 7
 [Job #2] User: Jerry | Pages: 12
 [Job #3] User: Roy | Pages: 5

The printer starts working through the queue one job at a time.
  [Printer] Now printing Job #1 for Carl (7 pages)... Done.

While the printer was busy, there are more jobs arriving:
  [Printer] Job #4 from Ryo (3 pages) added to queue. Pending jobs: 3
  [Printer] Job #5 from Joaquin (8 pages) added to queue. Pending jobs: 4

  Current queue for the job (4 job(s)), front -> back:
 [Job #2] User: Jerry | Pages: 12
 [Job #3] User: Roy | Pages: 5
 [Job #4] User: Ryo | Pages: 3
 [Job #5] User: Joaquin | Pages: 8

The printer keeps going until the queue is empty:

-- Printer processing all pending jobs (FIFO) --
  [Printer] Now printing Job #2 for Jerry (12 pages)... Done.
  [Printer] Now printing Job #3 for Roy (5 pages)... Done.
  [Printer] Now printing Job #4 for Ryo (3 pages)... Done.
  [Printer] Now printing Job #5 for Joaquin (8 pages)... Done.
-- All jobs processed. Printer is idle. --

One more job arrives after everything cleared out:
  [Printer] Job #6 from Kylian (2 pages) added to queue. Pending jobs: 1

-- Printer processing all pending jobs (FIFO) --
  [Printer] Now printing Job #6 for Kylian (2 pages)... Done.
-- All jobs processed. Printer is idle. --
```

OUTPUT

Why did I use Arrays?	Using a **circular array queue fits a print spooler perfectly because print jobs natively follow a First-In, First-Out (FIFO) model** which **the first document sent is the first one printed**, **matching the real-world expectation of a job application, first-come, first-served line**. By splitting the design into two header files, **classJob** acts as a clean data model for storing and formatting details like job IDs, user names, and page counts, while **classPrinter** houses the core queue logic and management methods. Setting a **fixed limit of 100 pending jobs makes a static array is an ideal choice for me for delivering much better memory performance than a linked list**. Instead of making constant calls to heap memory for every new node and wasting space on pointer addresses, **a circular array reserves one solid, contiguous block of memory up front**. Plus, **wrapping index positions with simple modulo operations ((idx + 1) % CAPACITY)** that **gives constant-time performance for adding and removing jobs without ever needing to shift elements in memory** which **giving all the speed of a linked list with none of the memory leaks**.

CODE AND OUTPUT ANALYSIS	In the code, I've used the Circular Based Array Queue because it is FIFO which in the queues in Jobs, its always First Come First Served which is like for Circular Based Array Queues like what I do in ILO B2. I've created two header files, classJob and classPrinter header file which the classJob is for the output and input operations for the Job Description like username, ID, and number of pages that will be printed. The functions for the classJob are there are **two constructors handle initialization** which is a **default constructor that fills in safe placeholder values**, and **a parameterized constructor that assigns specific details when a new job is created**. The three getter functions **(getId, getUserName, and getNumPages)** safely **expose those private details to other parts of your code without allowing them to be accidentally modified**. Finally, the **print() method** neatly formats all of this information into a human-readable line sent straight to the console **(formatted like [Job #1] User: Alice | Pages: 5)**, making it easy to see the status of any job in the queue at a glance. The classPrinter is the implementation of the Fixed Size Circular Array Queue which it allows the **Printer** class to reuse array slots efficiently without shifting existing elements down every time a print job finishes, keeping both enqueue **(addJob)** and dequeue **(processNextJob)** operations efficiently and also where the functions for the Printer to work is in here. Also, I've used the **Inline Functions** so that the **compiler to copy-paste the function's class body directly into the place where it was called**. Now, First functions, the constructor sets up the initial indices and job counter, while the state checker functions which are **isEmpty(), isFull(), and pendingCount()** which simply inspect the **count** variable to answer quick status questions. When you call **addJob() which is like Enqueue**, it checks for available room, creates a new job with an auto-incremented ID at the **rearIdx**, advances that index in a circle using **(rearIdx + 1) % CAPACITY**, and logs the update. Conversely, **processNextJob() like Dequeue** handles the oldest job at **frontIdx**, advances the front pointer around the circle using the same modulo wrap-around logic, and lowers the count, which initiates the function **processAllJobs()** which repeatedly triggers in a

	loop until the queue is completely drained. Lastly, **showQueue()** non-destructively walks through the array starting from **frontIdx** for **count** step which also leveraging modulo arithmetic to wrap around the array bound so you get a clean front-to-back preview of pending jobs without disrupting the queue. The output analysis is **like a printer but its organized now** because **it acts like a real-time activity log that confirms job submissions**, **warning when capacity of (100 jobs) is reached**, **listing active queues from front-to-back like Queues**, and **displaying step-by-step progress as jobs print on the printer until the system returns to an idle state**.

Conclusion Learnings	The lessons I have learned was **implementing STL queues, custom linked list queues, and array-based circular queues**. I learned how **First-In, First-Out (FIFO) data structures govern order-dependent systems like printer spoolers**, with circular modulo indexing **((idx + 1) % CAPACITY)** providing constant-time enqueuing and dequeuing without the performance cost of shifting elements in memory. In the Procedure, I've learned on how to modular approach across custom header files **(classJob.h and classPrinter.h)** and standard library implementations: **classJob** encapsulated individual print data through parameterized constructors and getter methods, **classPrinter** managed queue operations inline using circular array indices alongside state indicators **(isEmpty, isFull)**, and **auxiliary display mechanisms such as non-destructive loop traversals and temporary STL copy popping which safely previewed queued items without corrupting the underlying sequence**. In the Supplementary, I learned that applying the **circular queue code structure to a real-world print spooler simulation demonstrated the practical advantage of contiguous static memory over linked lists, proving how setting a fixed capacity (100 jobs) eliminates pointer overhead** and **provides a structured, real-time activity log that tracks job submissions which enforces memory capacity bounds and processes pending print tasks in a clean, sequential order**. I think I need to improve **my circular queue array and linked list queue structure** because **there are different approach on it and I need to practice to code it well and organized for the code to be formatted and operated for my desired behavior of the code that do not consume amounts of memory**.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **NONE**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **NONE**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.